



T.C. İSTANBUL KÜLTÜR ÜNİVERSİTESİ

COM5005 – WEB PROGRAMMING

LAB 12 – Authentication and Security

After completing this Lab, you will be able to

- Define Authentication
- Define Security using User Login

PROCEDURE 1 – Creating a New Project and Define Authentication

Authentication allows us to check whether there is a user logged into the system or not, and to perform session control, such as if there is a user logged into the system, they can browse some links but not others.

Step 1 – Create a new empty MVC project. And Select ASP.NET Web Application (.NET Framework).

Right click on the model folder and add a new class. The class name will be “User.cs”. Add " `using System.ComponentModel.DataAnnotations;` " for request and validation.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.ComponentModel.DataAnnotations;

namespace Authentication.Models
{
    0 references
    public class User
    {
        0 references
        public int UserId { get; set; }

        [Required(ErrorMessage = "Username is required")]
        0 references
        public string Username { get; set; }

        [Required(ErrorMessage = "The email address is required")]
        [EmailAddress(ErrorMessage = "Invalid Email Address")]
        0 references
        public string Email { get; set; }

        [Required(ErrorMessage = "Password is required")]
        0 references
        public string Password { get; set; }
    }
}
```

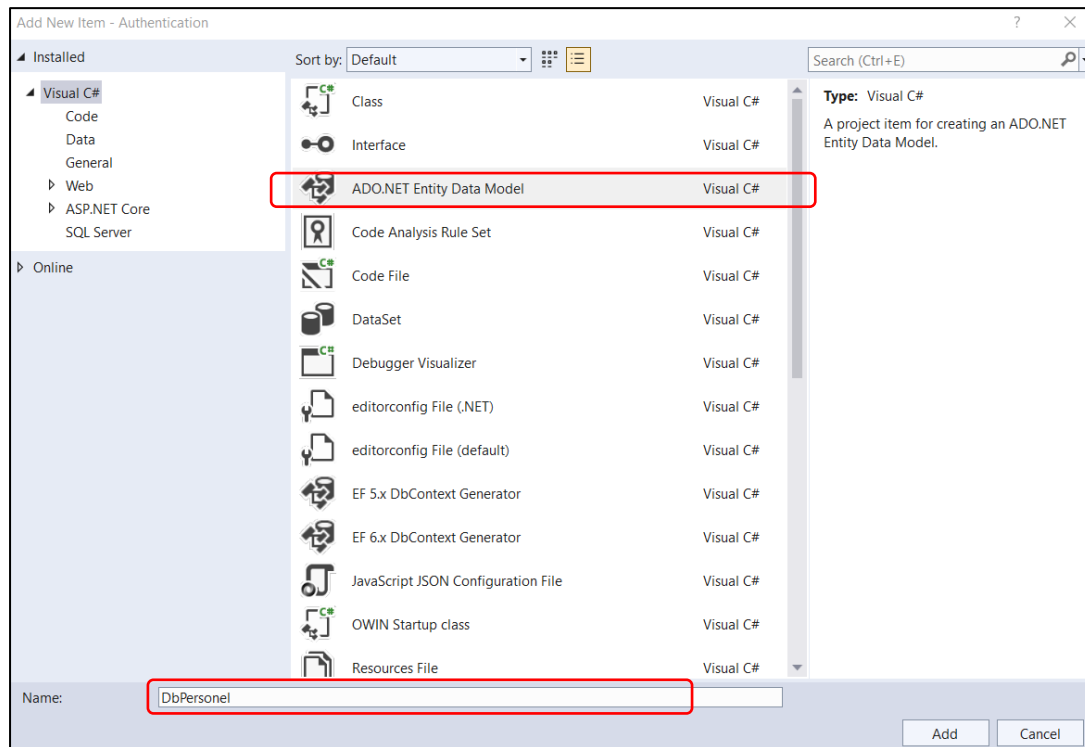
Step 2 – Right click on the Model folder and create ADO.NET Entity Data Model.

So we created a new database class named “DbPersonnel.cs”.

We integrate the class whose data we will use into the Database class.

For this we use the following code:

```
public System.Data.Entity.DbSet<Authentication.Models.User> Users { get; set; }
```



```
using System;
using System.Data.Entity;
using System.Linq;

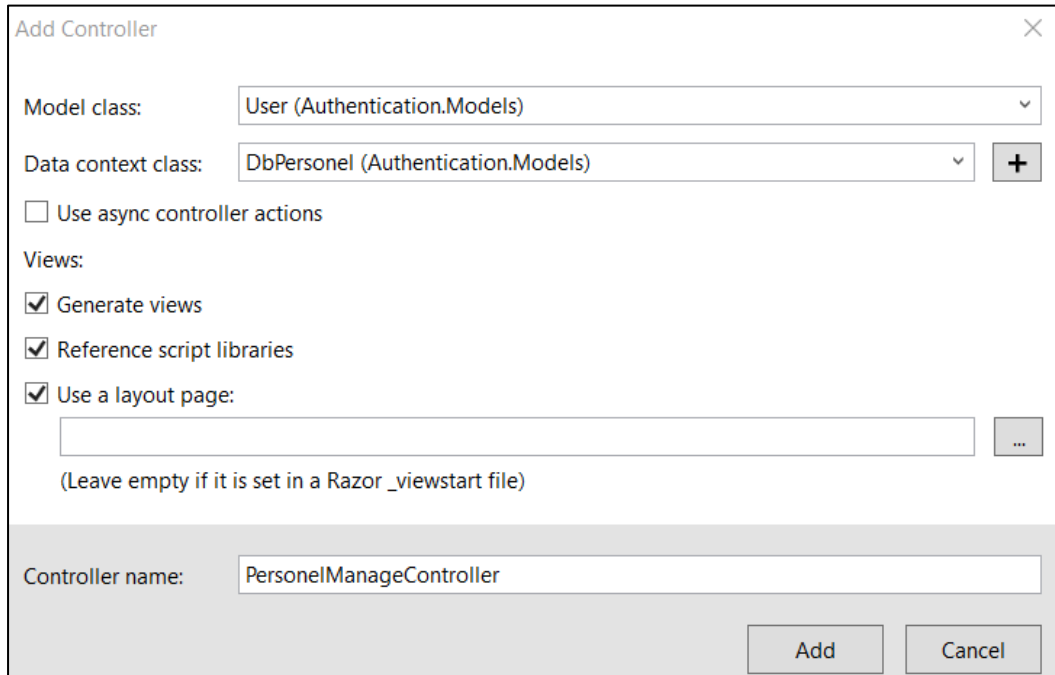
namespace Authentication.Models
{
    1 reference
    public class DbPersonel : DbContext
    {
        0 references
        public DbPersonel()
            : base("name=DbPersonel")
        {
        }
        0 references
        public System.Data.Entity.DbSet<Authentication.Models.User> Users { get; set; }
    }
}
```

Step 3 – After the database model, we design the Controller and Views.

First of all, we create a “PersonelManageController” that will connect to the database and manage personnel information.

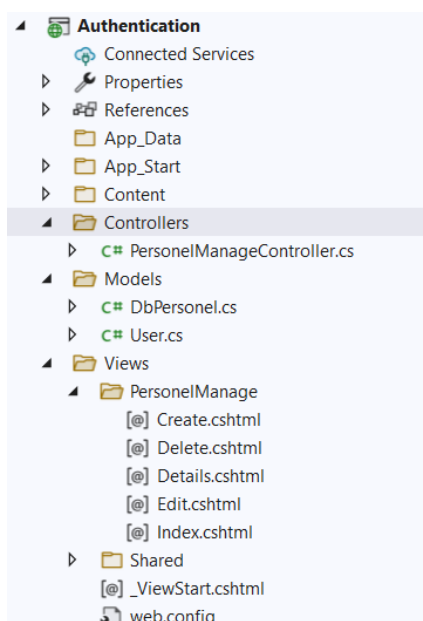
PersonelManageController will automatically create views using Entity Framework.

Right click on the Controller folder and add new controller, select Entity Framework.



The screenshot shows the 'Add Controller' dialog box. The 'Model class' is set to 'User (Authentication.Models)'. The 'Data context class' is set to 'DbPersonel (Authentication.Models)'. The 'Use async controller actions' checkbox is unchecked. Under the 'Views' section, the checkboxes for 'Generate views', 'Reference script libraries', and 'Use a layout page' are all checked. The 'Controller name' field contains 'PersonelManageController'. The 'Add' button is highlighted.

Step 4 – Pay attention to the View file, many web pages such as Create, Edit, Delete, Detail, Index.html are created automatically.



Step 5 – Authorization.

For example, we have come to any ActionResult and added [Authorize] to the top of it, now the user who is not logged in will not be able to see the view connected to this ActionResult and perform the actions. At the same time, we can write this Authorize on top of the Controller Class and set a session requirement for all ActionResult and Views connected to that Controller.

For example, let's authorize Index.html, other pages are public.

Open the PersonelManageController file. We add **[Authorize]** property to the controller of the web pages we want to authorize.

```
0 references
public class PersonelManageController : Controller
{
    private DbPersonel db = new DbPersonel();

    // GET: PersonelManage
    [Authorize] //Yetkilendirme //The user cannot access the view of the controller we wrote without logging in.
    public ActionResult Index()
    {
        return View(db.Users.ToList());
    }
}
```

Step 6 – Let's try to run the pages we have authorized. You will see that it gives an error because you are not authorized to view the page. Only registered users can access.

For this, we want it to redirect us to the Login page. If login is successful, we can access all pages.

Open the general **Web.config** file and write the following codes between the **<system.web>** tags.

```
<system.web>
  <compilation debug="true" targetFramework="4.7.2" />
  <httpRuntime targetFramework="4.7.2" />
  <authentication mode="Forms">
    <forms loginUrl="Security/Login"></forms>
  </authentication>
</system.web>
```

It writes a code that directs the Login action result method under the Security Controller. Then let's create Security controller for Login.

Step 7 – Let's right click on the controller and add a new controller named Security.

Let's create the Login Action Result method in the Security Controller.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;
using System.Web.Security;
using Authentication.Models;

namespace Authentication.Controllers
{
    0 references
    public class SecurityController : Controller
    {
        DbPersonel db = new DbPersonel();

        [HttpGet]
        // GET: Security
        0 references
        public ActionResult Login()
        {
            return View();
        }

        [HttpPost]
        0 references
        public ActionResult Login(User user)
        {
            var userInDb = db.Users.FirstOrDefault(x => x.Username == user.Username && x.Password == user.Password); //Is it available in the incoming user db?

            if (userInDb != null)
            {
                FormsAuthentication.SetAuthCookie(userInDb.Username, false); //The user is now authenticated, so the pages can be browsed.
                return RedirectToAction("Index", "PersonelManage");
            }
            else
            {
                ViewBag.Message = "Invalid Username or Password";
                return View();
            }
        }
    }
}
```

If the user information is in the database, it will be directed to the Index page and can view all pages. If no, it continues to redirect to the login page again.

Step 8 – Right click on the Security file under the View folder and add a new **Login.html** view.

It is a login page to login with Username and Password. If it matches the information in the database, index.html and other pages can be displayed. If the Username and Password do not match, new user registration can be obtained. The user is directed to the Create.html page by clicking the Register link.

Step 9 – Logout. Logout action link can be added to user pages.

```
0 references
public ActionResult Logout()
{
    FormsAuthentication.SignOut(); //We logged out the user
    return RedirectToAction("Login");
}
```

@Html.ActionLink("Logout", "Logout", "Security")

T.A. Merve GÜN

SecurityController:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;
using System.Web.Security;
using Authentication.Models;

namespace Authentication.Controllers
{
    public class SecurityController : Controller
    {
        DbPersonel db = new DbPersonel();

        [HttpGet]
        // GET: Security
        public ActionResult Login()
        {
            return View();
        }

        [HttpPost]
        public ActionResult Login(User user)
        {
            var userInDb = db.Users.FirstOrDefault(x => x.Username == user.Username && x.Password == user.Password); //Is it available in the incoming user db?

            if (userInDb != null)
            {
                FormsAuthentication.SetAuthCookie(userInDb.Username, false); //The user is now authenticated, so the pages can be browsed.
                return RedirectToAction("Index", "PersonelManage");
            }
            else
            {
                ViewBag.Message = "Invalid Username or Password";
                return View();
            }
        }
    }
}
```

Login.cshtml:

```
@model Authentication.Models.User
@{
    Layout = null;
}

<!DOCTYPE html>
<html>
<head>
    <meta name="viewport" content="width=device-width" />
    <title>Login</title>
</head>
<body>
    <div id="login">
        <h3 class="text-center text-white pt-5">Login form</h3>
        <div class="container">
            <div id="login-row" class="row justify-content-center align-items-center">
                <div id="login-column" class="col-md-6">
                    <div id="login-box" class="col-md-12">
                        @ViewBag.Message
                        @using (Html.BeginForm("Login", "Security", FormMethod.Post))
                        {
                            <h3 class="text-center text-info">Login</h3>
                            <div class="form-group">
                                @Html.LabelFor(m => m.Username)
                                @Html.TextBoxFor(m => m.Username, new { @class = "form-control", placeholder = "Username" })
                            </div>
                            <div class="form-group">
                                @Html.LabelFor(m => m.Password)
                                @Html.TextBoxFor(m => m.Password, new { @class = "form-control", placeholder = "Password" })
                            </div>
                            <div class="form-group">
                                <label for="remember-me" class="text-info"><span>Remember me</span>&nbsp;<span><input id="remember-me" name="remember-me"
type="checkbox"></span></label><br>
                                <button type="submit" name="submit" class="btn btn-info btn-md">Login</button>
                            </div>
                            <div id="register-link" class="text-right">
                                <a href=" ../PersonelManage/Create" class="text-info">Register here</a>
                            </div>
                        }
                    </div>
                </div>
            </div>
        </div>
    </div>
</body>
</html>
```